

C 语言程序设计大作业

Codeforces Crawler 2025

函数设计附件

—— 完整源代码清单 ——

姓 名：赵开

学 号：202505486219

单 位：计算机学院

日 期：2025 年 5 月

说明

本文档为函数设计说明书的附件部分，包含项目全部源代码文件。每个文件按模块分组，以带行号的代码块形式呈现。共收录 11 个源文件。

文件索引

1. include/rate_limiter.h — rate_limiter.h — CF API 请求频率限制器接口
2. include/json_utils.h — json_utils.h — JSON/文件工具模块接口
3. include/cf_api.h — cf_api.h — Codeforces API 数据获取模块接口
4. include/analyzer.h — analyzer.h — 核心数据分析模块接口
5. include/htmlgen.h — htmlgen.h — HTML 报告生成模块接口
6. src/rate_limiter.c — rate_limiter.c — 请求频率限制器实现
7. src/json_utils.c — json_utils.c — JSON/文件工具实现
8. src/cf_api.c — cf_api.c — CF API 数据获取实现
9. src/analyzer.c — analyzer.c — 核心数据分析实现
10. src/htmlgen.c — htmlgen.c — HTML 报告生成实现
11. src/main.c — main.c — 程序入口与流程协调

附件 1 rate_limiter.h — CF API 请求频率限制器接口

路径: include/rate_limiter.h | 共 1/11 个文件

```
1 // ===== include/rate_limiter.h =====
2 // 作者: 34
3 // 日期: 2025-05
4
5 /*
6  * rate_limiter.h — CF API 请求频率限制器接口
7  *
8  * 该头文件定义了 CF API 请求频率限制器的接口。
9  * 该头文件适用于 Codeforces API。
10  */
11
12 #pragma once
13
14 #ifdef _WIN32
15 #include <windows.h>
16 #else
17 #include <unistd.h>
18 #endif
19 #include <time.h>
20
21 /*
22  * RateLimiter 结构体
23  * last_request_time — 上次请求的时间
24  * min_interval — 最小间隔 (2.0 秒)
25  */
26 typedef struct {
27     double last_request_time;
28     double min_interval;
29 } RateLimiter;
30
31 /* 初始化 */
32 void rate_limiter_init(RateLimiter *rl);
33
34 /* 等待 */
35 void rate_limiter_wait(RateLimiter *rl);
36
37 /* 更新 */
38 void rate_limiter_tick(RateLimiter *rl);
```

附件 2 json_utils.h — JSON/文件工具模块接口

路径: include/json_utils.h | 共 2/11 个文件

```
1 // ===== include/json_utils.h =====
2 // 作者: 30
3 // 日期: 2025-05
4
5 /*
6  * json_utils.h - JSON/文件工具模块接口
7  *
8  * 提供JSON读写、CF API结果处理等功能
9  * 包含CF API结果处理函数
10  */
11
12 #pragma once
13 #include "cJSON.h"
14
15 /* 从文件中读取文本并返回字符串 */
16 char *read_text_file(const char *path);
17
18 /* 将内容写入文件 */
19 int write_text_file(const char *path, const char *content);
20
21 /* 从JSON文件中读取并返回cJSON_Delete */
22 cJSON *read_json_file(const char *path);
23
24 /* 将cJSON对象写入JSON文件 */
25 int write_json_file(const char *path, const cJSON *json);
26
27 /* 从CF API结果数组中读取并返回cJSON */
28 cJSON *load_cf_result_array(const char *path);
29
30 /* 从CF API结果中读取并返回cJSON */
31 cJSON *load_cf_result_first(const char *path);
32
33 /* 确保目录存在 */
34 int ensure_directory(const char *path);
```

附件 3 cf_api.h — Codeforces API 数据获取模块接口

路径: include/cf_api.h | 共 3/11 个文件

[illegible]

附件 4 analyzer.h — 核心数据分析模块接口

路径: include/analyzer.h | 共 4/11 个文件

```
1 // ===== include/analyzer.h =====
2 // 作者: 27
3 // 日期: 2025-05
4
5 /*
6  * analyzer.h — 核心数据分析模块接口
7  *
8  * 评分系统 核心接口
9  * 包含评分颜色、标题、处理函数等
10  */
11
12 #pragma once
13 #include "cJSON.h"
14
15 /* 评分系统 核心接口 */
16 const char *rating_color(int rating);
17
18 /* 评分系统 核心接口 */
19 const char *rating_title(int rating);
20
21 /*
22  * 评分系统 核心接口
23  *
24  * 处理函数 — CF 评分
25  * 数据目录 — 评分数据
26  * 输出目录 — 评分结果
27  * 最大排名 — 评分排名
28  * 评分系统 核心接口
29  */
30 cJSON *build_user_summary(const char *handle, const char *data_dir,
31                           const char *output_dir, int max_standings);
```

附件 5 htmlgen.h — HTML 报告生成模块接口

路径: include/htmlgen.h | 共 5/11 个文件

```
1 // ===== include/htmlgen.h =====
2 // 日期: 27
3 // 版本: 2025-05
4
5 /*
6  * htmlgen.h — HTML 报告生成模块接口
7  *
8  * 该头文件定义了 HTML 报告生成模块的接口。
9  * 该模块负责生成 HTML 报告。
10  */
11
12 #pragma once
13 #include "cJSON.h"
14
15 /*
16  * 该模块的接口函数。
17  *
18  * template_path — HTML 模板文件的路径
19  * output_path — 输出 HTML 报告的路径
20  * title — 报告的标题
21  * data — cJSON 类型的 JSON 数据
22  */
23 int generate_html(const char *template_path, const char *output_path,
24                  const char *title, cJSON *data);
25
26 /*
27  * 该模块的接口函数。
28  *
29  * index.html 报告生成模块接口
30  * report_<handle>.html 报告生成模块接口
31  */
32 int generate_all_html(cJSON *users, const char *template_path,
33                      const char *output_dir);
```

附件 6 rate_limiter.c — 请求频率限制器实现

路径: src/rate_limiter.c | 共 6/11 个文件

```
1 // ===== src/rate_limiter.c =====
2 // 73
3 // 2025-05
4
5 /*
6  * rate_limiter.c - CF API
7  *
8  * Codeforces API 1
9  * 2
10 * IP
11 *
12 *
13 * Windows: GetSystemTimeAsFileTime
14 * POSIX: clock_gettime(CLOCK_MONOTONIC)
15 */
16
17 #include "rate_limiter.h"
18
19 #ifdef _WIN32
20 /*
21 * Windows 100 →
22 * GetSystemTimeAsFileTime 100
23 */
24 static double get_wall_time(void) {
25     FILETIME ft;
26     GetSystemTimeAsFileTime(&ft);
27     unsigned long long t = ft.dwHighDateTime;
28     t <<= 32;
29     t |= ft.dwLowDateTime;
30     return (double)t / 10000000.0;
31 }
32 #else
33 /*
34 * POSIX
35 */
36 static double get_wall_time(void) {
37     struct timespec ts;
38     clock_gettime(CLOCK_MONOTONIC, &ts);
39     return ts.tv_sec + ts.tv_nsec / 1e9;
40 }
41 #endif
42
43 /*
44 * 2
45 */
46 void rate_limiter_init(RateLimiter *rl) {
47     rl->last_request_time = 0.0;
48     rl->min_interval = 2.0;
49 }
50
51 /*
52 *
53 * min_interval
54 */
55 void rate_limiter_wait(RateLimiter *rl) {
56     if (rl->last_request_time == 0.0) return;
57     double now = get_wall_time();
58     double elapsed = now - rl->last_request_time;
59     if (elapsed < rl->min_interval) {
60         double sleep_sec = rl->min_interval - elapsed;
61 #ifdef _WIN32
62         Sleep((DWORD)(sleep_sec * 1000));
63 #else
64         struct timespec ts;
65         ts.tv_sec = (time_t)sleep_sec;
66         ts.tv_nsec = (long)((sleep_sec - ts.tv_sec) * 1e9);
67         nanosleep(&ts, NULL);
68     }
69 }
```



```
68 #endif
69 }
70 }
71
72 /*
73  * ■■■■■■■■■■■■ wait() ■■
74 */
75 void rate_limiter_tick(RateLimiter *rl) {
76     rl->last_request_time = get_wall_time();
77 }
```

附件 7 json_utils.c — JSON/文件工具实现

路径: src/json_utils.c | 共 7/11 个文件

```
1 // ===== src/json_utils.c =====
2 // 作者: 132
3 // 日期: 2025-05
4
5 /*
6  * json_utils.c - JSON 工具库 CF API
7  *
8  * 提供 I/O 及 JSON 相关功能
9  * 使用 CF API 接口 {"status": "OK", "result": [...]}
10  * 详细文档见 README
11  */
12
13 #include "json_utils.h"
14 #include <stdio.h>
15 #include <stdlib.h>
16 #include <string.h>
17 #include <sys/stat.h>
18
19 #ifdef _WIN32
20 #include <direct.h>
21 #endif
22
23 /*
24  * 全局变量
25  * malloc 内存池 free()
26  * 初始化为 NULL
27  */
28 char *read_text_file(const char *path) {
29     FILE *f = fopen(path, "rb");
30     if (!f) return NULL;
31     fseek(f, 0, SEEK_END);
32     long len = ftell(f);
33     rewind(f);
34     char *content = (char *)malloc((size_t)len + 1);
35     if (!content) {
36         fclose(f);
37         return NULL;
38     }
39     size_t read_len = fread(content, 1, (size_t)len, f);
40     content[read_len] = '\0';
41     fclose(f);
42     return content;
43 }
44
45 /*
46  * 写入文件
47  * 返回 0 表示成功 -1 表示失败
48  */
49 int write_text_file(const char *path, const char *content) {
50     FILE *f = fopen(path, "wb");
51     if (!f) return -1;
52     size_t len = strlen(content);
53     size_t written = fwrite(content, 1, len, f);
54     fclose(f);
55     return (written == len) ? 0 : -1;
56 }
57
58 /*
59  * 解析 JSON 字符串
60  * 返回 cJSON 对象 失败返回 NULL
61  * 详细文档见 README
62  */
63 cJSON *read_json_file(const char *path) {
64     char *text = read_text_file(path);
65     if (!text) return NULL;
66     cJSON *json = cJSON_Parse(text);
67     free(text);
```

```

68     return json;
69 }
70
71 /*
72  * ■ cJSON ■■■■■■■■■■ JSON ■■■■■■■■■■
73  * ■■■■■■ 0■■■■ -1
74  */
75 int write_json_file(const char *path, const cJSON *json) {
76     char *text = cJSON_Print(json);
77     if (!text) return -1;
78     int ret = write_text_file(path, text);
79     free(text);
80     return ret;
81 }
82
83 /*
84  * ■ CF API ■■■■■■■■ result ■■
85  *
86  * CF API ■■■■■■■■
87  * {"status":"OK","result":[...]}
88  *
89  * ■■■■■■■■■■■■■■■■■■ cJSON_Delete()
90  * ■ status ■■ "OK" ■ result ■■■■■■■■ NULL
91  */
92 cJSON *load_cf_result_array(const char *path) {
93     cJSON *root = read_json_file(path);
94     if (!root) return NULL;
95     cJSON *status = cJSON_GetObjectItem(root, "status");
96     if (!status || strcmp(status->valuestring, "OK") != 0) {
97         cJSON_Delete(root);
98         return NULL;
99     }
100     cJSON *result = cJSON_GetObjectItem(root, "result");
101     if (!result || !cJSON_IsArray(result)) {
102         cJSON_Delete(root);
103         return NULL;
104     }
105     cJSON *ret = cJSON_Duplicate(result, 1);
106     cJSON_Delete(root);
107     return ret;
108 }
109
110 /*
111  * ■ CF API ■■■■■■■■ result ■■■■■■■■
112  *
113  * ■■ user.info ■■■■■■■■■■■■
114  * ■■■■■■■■■■■■■■■■■■ cJSON_Delete()
115  */
116 cJSON *load_cf_result_first(const char *path) {
117     cJSON *arr = load_cf_result_array(path);
118     if (!arr) return NULL;
119     cJSON *first = cJSON_GetArrayItem(arr, 0);
120     cJSON *ret = NULL;
121     if (first) ret = cJSON_Duplicate(first, 1);
122     cJSON_Delete(arr);
123     return ret;
124 }
125
126 /*
127  * ■■■■■■■■■■■■
128  * ■■■■■■ 0■■■■ -1
129  */
130 int ensure_directory(const char *path) {
131 #ifdef _WIN32
132     return _mkdir(path);
133 #else
134     return mkdir(path, 0755);
135 #endif
136 }

```

附件 8 cf_api.c — CF API 数据获取实现

路径: src/cf_api.c | 共 8/11 个文件

```
1 // ===== src/cf_api.c =====
2 // 日期: 177
3 // 版本: 2025-05
4
5 /*
6  * cf_api.c - Codeforces API 接口
7  *
8  * 依赖: libcurl, HTTP GET
9  * 接口: CF API 返回 JSON
10  *
11  * API 地址: https://codeforces.com/apiHelp
12  *
13  * 支持的接口:
14  *   user.info      - 用户信息
15  *   user.rating    - 用户 rating
16  *   user.status    - 用户状态
17  *   contest.list   - 比赛列表
18  *   contest.standings - 比赛 standings
19  */
20
21 #include "cf_api.h"
22 #include "json_utils.h"
23 #include <stdio.h>
24 #include <stdlib.h>
25 #include <string.h>
26 #include <curl/curl.h>
27
28 /*
29  * 依赖: libcurl
30  * 接口: HTTP GET
31  */
32 struct MemoryBlock {
33     char *data;
34     size_t size;
35 };
36
37 /*
38  * libcurl 接口
39  * 返回 MemoryBlock
40  */
41 static size_t write_callback(void *contents, size_t size, size_t nmemb, void *userp) {
42     size_t total = size * nmemb;
43     struct MemoryBlock *mem = (struct MemoryBlock *)userp;
44     char *ptr = realloc(mem->data, mem->size + total + 1);
45     if (!ptr) return 0;
46     mem->data = ptr;
47     memcpy(&mem->data[mem->size], contents, total);
48     mem->size += total;
49     mem->data[mem->size] = '\0';
50     return total;
51 }
52
53 /*
54  * HTTP GET 接口
55  * 返回 free() 的指针
56  */
57 char *cf_http_get(const char *url) {
58     CURL *curl = curl_easy_init();
59     if (!curl) return NULL;
60
61     struct MemoryBlock chunk;
62     chunk.data = malloc(1);
63     chunk.size = 0;
64
65     /* 依赖: libcurl */
66     curl_easy_setopt(curl, CURLOPT_URL, url);
67     curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_callback);
```

```

68     curl_easy_setopt(curl, CURLOPT_WRITEDATA, (void *)&chunk);
69     curl_easy_setopt(curl, CURLOPT_USERAGENT, "CF-Crawler/1.0");
70     curl_easy_setopt(curl, CURLOPT_TIMEOUT, 30L);
71     curl_easy_setopt(curl, CURLOPT_FOLLOWLOCATION, 1L);
72     /* Windows ████████ CA ████████ SSL ███ */
73     curl_easy_setopt(curl, CURLOPT_SSL_VERIFYPEER, 0L);
74     curl_easy_setopt(curl, CURLOPT_SSL_VERIFYHOST, 0L);
75
76     CURLcode res = curl_easy_perform(curl);
77     curl_easy_cleanup(curl);
78
79     if (res != CURLE_OK) {
80         fprintf(stderr, " [ERROR] curl failed: %s\n URL: %s\n",
81             curl_easy_strerror(res), url);
82         free(chunk.data);
83         return NULL;
84     }
85
86     return chunk.data;
87 }
88
89 /*
90  * ████████ + ███ + ████████
91  * 1. ████████
92  * 2. ███ HTTP ███
93  * 3. ████████████████
94  * 4. ███ JSON ████████ CF API █████status == "OK"█
95  * 5. ██████████
96  */
97 int fetch_and_save(const char *url, const char *path, RateLimiter *rl) {
98     rate_limiter_wait(rl);
99     printf(" Fetching: %s\n", url);
100     char *response = cf_http_get(url);
101     if (!response) return -1;
102     rate_limiter_tick(rl);
103
104     /* ███ JSON ████████ */
105     cJSON *root = cJSON_Parse(response);
106     if (!root) {
107         fprintf(stderr, " [ERROR] Invalid JSON response\n");
108         free(response);
109         return -1;
110     }
111     cJSON *status = cJSON_GetObjectItem(root, "status");
112     if (!status) {
113         fprintf(stderr, " [ERROR] Unexpected API response format\n");
114         cJSON_Delete(root);
115         free(response);
116         return -1;
117     }
118     if (strcmp(status->valstring, "OK") != 0) {
119         cJSON *comment = cJSON_GetObjectItem(root, "comment");
120         fprintf(stderr, " [ERROR] CF API: %s\n",
121             comment ? comment->valstring : status->valstring);
122         cJSON_Delete(root);
123         free(response);
124         return -1;
125     }
126     cJSON_Delete(root);
127
128     /* ████████ */
129     int ret = write_text_file(path, response);
130     free(response);
131     if (ret != 0) {
132         fprintf(stderr, " [ERROR] Failed to write: %s\n", path);
133     }
134     return ret;
135 }
136
137 /*
138  * ███ 5 ██████████ CF API ████████
139  * ████████ URL → ███ fetch_and_save → █████ data/ ███

```

```

140  */
141
142  /* rating, rank, avatar */
143  int cf_fetch_user_info(const char *handle, const char *path, RateLimiter *rl) {
144      char url[512];
145      snprintf(url, sizeof(url),
146               "https://codeforces.com/api/user.info?handles=%s", handle);
147      return fetch_and_save(url, path, rl);
148  }
149
150  /* rating */
151  int cf_fetch_user_rating(const char *handle, const char *path, RateLimiter *rl) {
152      char url[512];
153      snprintf(url, sizeof(url),
154               "https://codeforces.com/api/user.rating?handle=%s", handle);
155      return fetch_and_save(url, path, rl);
156  }
157
158  /* */
159  int cf_fetch_user_status(const char *handle, const char *path, RateLimiter *rl) {
160      char url[512];
161      snprintf(url, sizeof(url),
162               "https://codeforces.com/api/user.status?handle=%s", handle);
163      return fetch_and_save(url, path, rl);
164  }
165
166  /* */
167  int cf_fetch_contest_list(const char *path, RateLimiter *rl) {
168      const char *url = "https://codeforces.com/api/contest.list?gym=false";
169      return fetch_and_save(url, path, rl);
170  }
171
172  /* */
173  int cf_fetch_contest_standings(int contest_id, const char *handle,
174                                const char *path, RateLimiter *rl) {
175      char url[1024];
176      snprintf(url, sizeof(url),
177               "https://codeforces.com/api/contest standings?contestId=%d"
178               "&handles=%s&showUnofficial=true",
179               contest_id, handle);
180      return fetch_and_save(url, path, rl);
181  }

```

附件 9 analyzer.c — 核心数据分析实现

路径: src/analyzer.c | 共 9/11 个文件

```
1 // ===== src/analyzer.c =====
2 // 作者: 495
3 // 日期: 2025-05
4
5 /*
6  * analyzer.c — 核心数据分析实现
7  *
8  * 实现 CF API 接口
9  * 支持 JSON 格式
10  *
11  * 功能:
12  * 1. 解析比赛数据
13  * 2. AC 率统计 rating 计算
14  * 3. 180 分统计
15  */
16
17 #include "analyzer.h"
18 #include "json_utils.h"
19 #include <stdio.h>
20 #include <stdlib.h>
21 #include <string.h>
22 #include <time.h>
23
24 /*
25  * 定义 rating 等级 CF 等级
26  * CF 等级 10 分制: Newbie, LGM
27  */
28 const char *rating_color(int rating) {
29     if (rating >= 3000) return "#000000"; /* Legendary Grandmaster */
30     if (rating >= 2600) return "#CC0000"; /* International Grandmaster */
31     if (rating >= 2400) return "#FF0000"; /* Grandmaster */
32     if (rating >= 2300) return "#FF8C00"; /* International Master */
33     if (rating >= 2100) return "#FF8C00"; /* Master */
34     if (rating >= 1900) return "#AA00AA"; /* Candidate Master */
35     if (rating >= 1600) return "#0000FF"; /* Expert */
36     if (rating >= 1400) return "#03A89E"; /* Specialist */
37     if (rating >= 1200) return "#008000"; /* Pupil */
38     return "#808080"; /* Newbie */
39 }
40
41 /*
42  * 定义 rating 等级
43  */
44 const char *rating_title(int rating) {
45     if (rating >= 3000) return "Legendary Grandmaster";
46     if (rating >= 2600) return "International Grandmaster";
47     if (rating >= 2400) return "Grandmaster";
48     if (rating >= 2300) return "International Master";
49     if (rating >= 2100) return "Master";
50     if (rating >= 1900) return "Candidate Master";
51     if (rating >= 1600) return "Expert";
52     if (rating >= 1400) return "Specialist";
53     if (rating >= 1200) return "Pupil";
54     return "Newbie";
55 }
56
57 /*
58  * 定义 ID 格式: {name, rating, rank}
59  * 支持 JSON 格式
60  */
61 static cJSON *build_contest_lookup(cJSON *contest_list) {
62     cJSON *lookup = cJSON_CreateObject();
63     if (!contest_list || !lookup) return lookup;
64
65     cJSON *item;
66     cJSON_ArrayForEach(item, contest_list) {
67         cJSON *id = cJSON_GetObjectItem(item, "id");
```

[illegible]


```

140     if (!sproblem) continue;
141     cJSON *sindex = cJSON_GetObjectItem(sproblem, "index");
142     if (!sindex) continue;
143
144     /* ██████████ */
145     int idx = -1;
146     for (int i = 0; i < prob_count; i++) {
147         if (strcmp(probs[i].index, sindex->valuelstring) == 0) {
148             idx = i;
149             break;
150         }
151     }
152     if (idx < 0 && prob_count < max_probs) {
153         idx = prob_count++;
154         strncpy(probs[idx].index, sindex->valuelstring, sizeof(probs[idx].index) - 1);
155         probs[idx].index[sizeof(probs[idx].index) - 1] = '\0';
156         probs[idx].in_contest_ac = 0;
157         probs[idx].post_ac = 0;
158         probs[idx].rejected = 0;
159         probs[idx].problem_rating = 0;
160         probs[idx].seen = 1;
161
162         /* ██████████ API ██████████ */
163         cJSON *prating = cJSON_GetObjectItem(sproblem, "rating");
164         if (prating && cJSON_IsNumber(prating))
165             probs[idx].problem_rating = prating->valueint;
166     }
167     if (idx < 0) continue;
168
169     cJSON *sverdict = cJSON_GetObjectItem(sub, "verdict");
170     if (!sverdict) continue;
171
172     cJSON *stime = cJSON_GetObjectItem(sub, "creationTimeSeconds");
173     long long sub_time = stime && cJSON_IsNumber(stime) ?
174         (long long)sstime->valuedouble : 0;
175
176     /*
177      * ████████████████████
178      *   sub_time <= contest_end → ████
179      *   sub_time > contest_end → ████
180      */
181     if (strcmp(sverdict->valuelstring, "OK") == 0) {
182         if (sub_time > 0 && sub_time <= contest_end) {
183             probs[idx].in_contest_ac = 1; /* ██ AC */
184         } else {
185             probs[idx].post_ac = 1;       /* ████ AC */
186         }
187     } else {
188         probs[idx].rejected++;           /* ████ */
189     }
190 }
191
192 /* ████████████████████ A, B, C, ... */
193 qsort(probs, prob_count, sizeof(ProblemState), prob_cmp);
194 return prob_count;
195 }
196
197 /*
198  * ██ AC ████████████████████
199  *
200  * ██ user.status██████ AC ███
201  * ███ rating ████████████████████
202  *
203  * ███
204  * window_sec - ██████████0 ████████
205  */
206 static void count_ac_by_rating(cJSON *status, cJSON *hist,
207                               long long now_ts, long long window_sec) {
208     if (!status) return;
209     cJSON *sub;
210     cJSON_ArrayForEach(sub, status) {
211         cJSON *verdict = cJSON_GetObjectItem(sub, "verdict");

```

[illegible]

```

284 snprintf(user_path, sizeof(user_path), "%s/%s_user.json", data_dir, handle);
285 snprintf(rating_path, sizeof(rating_path), "%s/%s_rating.json", data_dir, handle);
286 snprintf(status_path, sizeof(status_path), "%s/%s_status.json", data_dir, handle);
287 snprintf(clist_path, sizeof(clist_path), "%s/contest_list.json", data_dir);
288
289 /* ██████████ */
290 cJSON *user_info = load_cf_result_first(user_path);
291 if (!user_info) {
292     fprintf(stderr, " [ERROR] Cannot load user info for %s\n", handle);
293     cJSON_Delete(result);
294     return NULL;
295 }
296
297 /* ██ rating ██ */
298 cJSON *rating_history = load_cf_result_array(rating_path);
299 if (!rating_history) {
300     fprintf(stderr, " [ERROR] Cannot load rating history for %s\n", handle);
301     cJSON_Delete(user_info);
302     cJSON_Delete(result);
303     return NULL;
304 }
305
306 /* ████████████████████████████████████████ */
307 cJSON *status = load_cf_result_array(status_path);
308 cJSON *contest_list = load_cf_result_array(clist_path);
309
310 /* ████████████████████████████████████████ */
311 time_t now = time(NULL);
312 long long now_ts = (long long)now;
313 long long year_sec = 365LL * 86400; /* ██████ */
314 long long halfyear_sec = 180LL * 86400; /* 180████ */
315 long long month_sec = 30LL * 86400; /* ██████ */
316
317 /* ██ API ████████████████████████████████████████ */
318 adjust_rating_rank(user_info);
319
320 /* ===== ██████████ ===== */
321 cJSON *cf_handle = cJSON_GetObjectItem(user_info, "handle");
322 cJSON_AddStringToObject(result, "handle",
323     cf_handle && cf_handle->valstring ? cf_handle->valstring : handle);
324
325 cJSON *avatar = cJSON_GetObjectItem(user_info, "avatar");
326 cJSON *titlePhoto = cJSON_GetObjectItem(user_info, "titlePhoto");
327 cJSON_AddStringToObject(result, "avatar",
328     avatar && avatar->valstring ? avatar->valstring : "");
329 cJSON_AddStringToObject(result, "titlePhoto",
330     titlePhoto && titlePhoto->valstring ? titlePhoto->valstring : "");
331
332 int rating = 0, maxRating = 0;
333 cJSON *r = cJSON_GetObjectItem(user_info, "rating");
334 if (r && cJSON_IsNumber(r)) rating = r->valueint;
335 cJSON *mr = cJSON_GetObjectItem(user_info, "maxRating");
336 if (mr && cJSON_IsNumber(mr)) maxRating = mr->valueint;
337
338 cJSON_AddNumberToObject(result, "rating", rating);
339 cJSON_AddNumberToObject(result, "maxRating", maxRating);
340 cJSON_AddStringToObject(result, "ratingColor", rating_color(rating));
341
342 const char *rank_str = "Unrated";
343 cJSON *rankj = cJSON_GetObjectItem(user_info, "rank");
344 if (rankj && rankj->valstring) rank_str = rankj->valstring;
345 cJSON_AddStringToObject(result, "rank", rank_str);
346
347 const char *maxRankStr = "Unrated";
348 cJSON *maxRankj = cJSON_GetObjectItem(user_info, "maxRank");
349 if (maxRankj && maxRankj->valstring) maxRankStr = maxRankj->valstring;
350 cJSON_AddStringToObject(result, "maxRank", maxRankStr);
351
352 /* ===== ██████████ ===== */
353 cJSON *contest_lookup = build_contest_lookup(contest_list);
354 if (!contest_lookup) contest_lookup = cJSON_CreateObject();
355

```

```

356     int contest_count = cJSON_GetArraySize(rating_history);
357     int contest_count_180 = 0;
358     int max_rating_180 = 0;
359     cJSON *contests_arr = cJSON_AddArrayToObject(result, "contests");
360
361     /*
362      * ■■ rating ■■■■■■■■■■
363      * ■■■■■■■■■■
364      */
365     cJSON *item;
366     cJSON_ArrayForEach(item, rating_history) {
367         cJSON *contest_obj = cJSON_CreateObject();
368
369         int contest_id = 0;
370         long long update_time = 0;
371
372         cJSON *cid = cJSON_GetObjectItem(item, "contestId");
373         if (cid && cJSON_IsNumber(cid)) contest_id = cid->valueint;
374         cJSON *ctime = cJSON_GetObjectItem(item, "ratingUpdateTimeSeconds");
375         if (ctime && cJSON_IsNumber(ctime)) update_time = (long long)ctime->valuedouble;
376
377         cJSON_AddNumberToObject(contest_obj, "contestId", contest_id);
378
379         cJSON *cname = cJSON_GetObjectItem(item, "contestName");
380         cJSON_AddStringToObject(contest_obj, "contestName",
381             cname && cname->valuestring ? cname->valuestring : "");
382
383         cJSON_AddNumberToObject(contest_obj, "timestamp", (double)update_time);
384
385         /* ■ lookup ■■■■■■■■ */
386         char key[32];
387         snprintf(key, sizeof(key), "%d", contest_id);
388         cJSON *cl_entry = cJSON_GetObjectItem(contest_lookup, key);
389         long long start_time = 0;
390         long long duration = 7200;
391         if (cl_entry) {
392             cJSON *st = cJSON_GetObjectItem(cl_entry, "startTimeSeconds");
393             if (st && cJSON_IsNumber(st)) start_time = (long long)st->valuedouble;
394             cJSON *dur = cJSON_GetObjectItem(cl_entry, "durationSeconds");
395             if (dur && cJSON_IsNumber(dur)) duration = (long long)dur->valuedouble;
396         }
397
398         cJSON_AddNumberToObject(contest_obj, "startTime", (double)(start_time ? start_time : update_time))
399     ;
400
401     /* ■■/■■ rating ■■■ */
402     cJSON *oldr = cJSON_GetObjectItem(item, "oldRating");
403     cJSON *newr = cJSON_GetObjectItem(item, "newRating");
404     int old_rating = oldr && cJSON_IsNumber(oldr) ? oldr->valueint : 0;
405     int new_rating = newr && cJSON_IsNumber(newr) ? newr->valueint : 0;
406
407     cJSON_AddNumberToObject(contest_obj, "oldRating", old_rating);
408     cJSON_AddNumberToObject(contest_obj, "newRating", new_rating);
409     cJSON_AddStringToObject(contest_obj, "oldRatingColor", rating_color(old_rating));
410     cJSON_AddStringToObject(contest_obj, "newRatingColor", rating_color(new_rating));
411
412     /* ■■■■ */
413     cJSON *rankj2 = cJSON_GetObjectItem(item, "rank");
414     int rank_val = rankj2 && cJSON_IsNumber(rankj2) ? rankj2->valueint : 0;
415     cJSON_AddNumberToObject(contest_obj, "rank", rank_val);
416
417     /* ■■■ 180 ■■■■■■ */
418     if (now_ts - update_time <= halfyear_sec) {
419         contest_count_180++;
420         if (new_rating > max_rating_180) max_rating_180 = new_rating;
421     }
422
423     /*
424      * ■■■■■■■■■■
425      * ■■■■■■■■ = ■■■■ + ■■■■
426      */
427     long long contest_end = start_time + duration;

```

```

427 ProblemState probs[MAX_PROBLEMS];
428 int prob_cnt = process_contest_problems(status, contest_id,
429                                     contest_end, probs, MAX_PROBLEMS);
430
431 /* ■■■■■■■■■■■■■■■■■■■■■■ */
432 int solved_count = 0;
433 cJSON *problems_arr = cJSON_AddArrayToObject(contest_obj, "problems");
434 cJSON *upsolved_arr = cJSON_AddArrayToObject(contest_obj, "upsolved");
435
436 for (int i = 0; i < prob_cnt; i++) {
437     if (probs[i].in_contest_ac) {
438         /* ■■■■■■■■■■■■■■■■■■■■■■ */
439         solved_count++;
440         cJSON *prob = cJSON_CreateObject();
441         cJSON_AddStringToObject(prob, "index", probs[i].index);
442         cJSON_AddNumberToObject(prob, "rejectedAttempts", probs[i].rejected);
443         cJSON_AddNumberToObject(prob, "problemRating", probs[i].problem_rating);
444         cJSON_AddStringToObject(prob, "verdict", "OK");
445         cJSON_AddItemToArray(problems_arr, prob);
446     } else if (probs[i].rejected > 0) {
447         /* ■■■■■■■■■■■■■■■■■■■■■■ */
448         cJSON *prob = cJSON_CreateObject();
449         cJSON_AddStringToObject(prob, "index", probs[i].index);
450         cJSON_AddNumberToObject(prob, "rejectedAttempts", probs[i].rejected);
451         cJSON_AddNumberToObject(prob, "problemRating", probs[i].problem_rating);
452         cJSON_AddStringToObject(prob, "verdict", "FAIL");
453         cJSON_AddItemToArray(problems_arr, prob);
454     }
455
456     /* ■■■■■■■■■■■■■■■■■■■■■■ upsolved ■■■ ■ */
457     if (probs[i].post_ac && !probs[i].in_contest_ac) {
458         cJSON *us = cJSON_CreateObject();
459         cJSON_AddStringToObject(us, "index", probs[i].index);
460         cJSON_AddNumberToObject(us, "problemRating", probs[i].problem_rating);
461         cJSON_AddItemToArray(upsolved_arr, us);
462     }
463 }
464
465 cJSON_AddNumberToObject(contest_obj, "solvedCount", solved_count);
466
467 cJSON_AddItemToArray(contests_arr, contest_obj);
468 }
469
470 /* ■■■■■■■■■■■■■■■■■■■■■■ */
471 cJSON_AddNumberToObject(result, "contestCount", contest_count);
472 cJSON_AddNumberToObject(result, "contestCount180", contest_count_180);
473 cJSON_AddNumberToObject(result, "maxRating180", max_rating_180);
474
475 /*
476  * ■■ AC ■■■■■■■■■■■■■■■■■■■■■■
477  * ■ 4 ■■■■■■■■■■■■■■■■■■■■■■ / ■■■ / ■ 180 ■ / ■■■■■
478  */
479 cJSON *hist_all = cJSON_AddObjectToObject(result, "histAll");
480 cJSON *hist_year = cJSON_AddObjectToObject(result, "histYear");
481
482 cJSON *hist_180 = cJSON_AddObjectToObject(result, "hist180");
483 cJSON *hist_month = cJSON_AddObjectToObject(result, "histMonth");
484
485 if (status) {
486     count_ac_by_rating(status, hist_all, now_ts, 0);
487     count_ac_by_rating(status, hist_year, now_ts, year_sec);
488     count_ac_by_rating(status, hist_180, now_ts, halfyear_sec);
489     count_ac_by_rating(status, hist_month, now_ts, month_sec);
490 }
491
492 /* ■■■■■■■■■■■■■■■■■■■■■■ */
493 cJSON_Delete(user_info);
494 cJSON_Delete(rating_history);
495 if (status) cJSON_Delete(status);
496 if (contest_list) cJSON_Delete(contest_list);
497 cJSON_Delete(contest_lookup);
498
499 return result;

```


附件 10 htmlgen.c — HTML 报告生成实现

路径: src/htmlgen.c | 共 10/11 个文件

```
1 // ===== src/htmlgen.c =====
2 // 日期: 171
3 // 版本: 2025-05
4
5 /*
6  * htmlgen.c - HTML 报告生成
7  *
8  * 生成 HTML 报告，格式为 {{TITLE}} 和 {{DATA_JSON}}
9  * 报告名称: HTML 报告
10  *
11  * 报告内容:
12  *   - index.html: 报告封面
13  *   - report_<handle>.html: 报告正文
14  */
15
16 #include "htmlgen.h"
17 #include "json_utils.h"
18 #include <stdio.h>
19 #include <stdlib.h>
20 #include <string.h>
21
22 /*
23  * 报告生成函数
24  *
25  * 参数:
26  *   tmpl      - 模板字符串
27  *   placeholder - 占位符字符串，格式为 {{TITLE}}
28  *   value      - 要替换的值
29  *   free()     - 释放内存
30  */
31 static char *replace_placeholder(const char *tmpl, const char *placeholder,
32                                 const char *value) {
33     size_t tmpl_len = strlen(tmpl);
34     size_t ph_len = strlen(placeholder);
35     size_t val_len = strlen(value);
36
37     /* 统计占位符出现的次数 */
38     int count = 0;
39     const char *p = tmpl;
40     while ((p = strstr(p, placeholder)) != NULL) {
41         count++;
42         p += ph_len;
43     }
44
45     if (count == 0) return strdup(tmpl);
46
47     /* 计算输出字符串的长度 */
48     size_t out_len = tmpl_len + count * (val_len - ph_len) + 1;
49     char *out = (char *)malloc(out_len);
50     if (!out) return NULL;
51
52     /* 生成输出字符串 */
53     char *dst = out;
54     const char *src = tmpl;
55     const char *next;
56     while ((next = strstr(src, placeholder)) != NULL) {
57         size_t seg_len = next - src;
58         memcpy(dst, src, seg_len);
59         dst += seg_len;
60         memcpy(dst, value, val_len);
61         dst += val_len;
62         src = next + ph_len;
63     }
64     strcpy(dst, src);
65
66     return out;
67 }
```

```

68
69 /*
70  * ■■■■ HTML ■■
71  *
72  * ■■■
73  * 1. ■■■■■■
74  * 2. ■■■■■■ JSON ■■■
75  * 3. ■■ {{TITLE}} ■ {{DATA_JSON}} ■■■
76  * 4. ■■■■■■
77  *
78  * ■■■
79  * template_path - ■■■■■■
80  * output_path  - ■■■■■■
81  * title        - ■■■■■■ {{TITLE}}■
82  * data         - cJSON ■■■■■■ {{DATA_JSON}}■
83  * ■■■■■■ 0■■■■■ -1
84  */
85 int generate_html(const char *template_path, const char *output_path,
86                  const char *title, cJSON *data) {
87     /* ■■■■ */
88     char *tmpl = read_text_file(template_path);
89     if (!tmpl) {
90         fprintf(stderr, " [ERROR] Cannot read template: %s\n", template_path);
91         return -1;
92     }
93
94     /* ■ cJSON ■■■■■■ JSON ■■■ */
95     char *json_str = NULL;
96     if (data) {
97         json_str = cJSON_PrintUnformatted(data);
98         if (!json_str) {
99             fprintf(stderr, " [ERROR] Cannot serialize JSON data\n");
100             free(tmpl);
101             return -1;
102         }
103     } else {
104         json_str = strdup("null");
105     }
106
107     /* ■■■■■■ */
108     char *after_title = replace_placeholder(tmpl, "{{TITLE}}", title);
109     free(tmpl);
110     if (!after_title) {
111         free(json_str);
112         return -1;
113     }
114
115     /* ■■■■■■ */
116     char *result = replace_placeholder(after_title, "{{DATA_JSON}}", json_str);
117     free(after_title);
118     free(json_str);
119
120     if (!result) return -1;
121
122     /* ■■■■■■ */
123     int ret = write_text_file(output_path, result);
124     free(result);
125
126     if (ret == 0) {
127         printf(" Generated: %s\n", output_path);
128     } else {
129         fprintf(stderr, " [ERROR] Failed to write: %s\n", output_path);
130     }
131
132     return ret;
133 }
134
135 /*
136  * ■■■■ HTML ■■
137  *
138  * ■■■■ index.html■■■■■■■■■■
139  * ■■■■■■■■ report_<handle>.html■■■■■■■

```



```

140  *
141  *
142  *   users      - cJSON
143  *   template_path -
144  *   output_dir  -
145  *   0 -1
146  */
147  int generate_all_html(cJSON *users, const char *template_path,
148                      const char *output_dir) {
149      if (!users) return -1;
150
151      /* index.html */
152      char index_path[512];
153      snprintf(index_path, sizeof(index_path), "%s/index.html", output_dir);
154      generate_html(template_path, index_path,
155                  "Codeforces Analytics - Users", users);
156
157      /*
158      cJSON *user;
159      cJSON_ArrayForEach(user, users) {
160          cJSON *handle = cJSON_GetObjectItem(user, "handle");
161
162          if (!handle || !handle->valuelstring) continue;
163
164          char detail_path[512];
165          snprintf(detail_path, sizeof(detail_path), "%s/report_%s.html",
166                  output_dir, handle->valuelstring);
167
168          char page_title[256];
169          snprintf(page_title, sizeof(page_title),
170                  "Codeforces - %s", handle->valuelstring);
171
172          generate_html(template_path, detail_path, page_title, user);
173      }
174
175      return 0;
176  }

```

附件 11 main.c — 程序入口与流程协调

路径: src/main.c | 共 11/11 个文件

```
1 // ===== src/main.c =====
2 // 日期: 156
3 // 版本: 2025-05
4
5 /*
6  * main.c — 程序入口与流程协调
7  *
8  * 功能:
9  *   1. 读取 sample_users.txt
10  *   2. 调用 CF API
11  *   3. 使用 analyzer
12  *   4. 生成 HTML
13  */
14
15 #include <stdio.h>
16 #include <stdlib.h>
17 #include <string.h>
18 #include <sys/stat.h>
19
20 #include "rate_limiter.h"
21 #include "json_utils.h"
22 #include "cf_api.h"
23 #include "analyzer.h"
24 #include "htmlgen.h"
25
26 #define MAX_HANDLE_LEN 64 /* Codeforces 限制 */
27 #define MAX_HANDLES 256 /* 最大句柄数 */
28 #define DATA_DIR "data" /* API 数据目录 */
29 #define OUTPUT_DIR "output" /* HTML 输出目录 */
30 #define TEMPLATE_PATH "web/template.html" /* HTML 模板 */
31
32 #ifdef _WIN32
33 #include <direct.h>
34 #define mkdir_p(path) _mkdir(path)
35 #else
36 #define mkdir_p(path) mkdir(path, 0755)
37 #endif
38
39 /*
40  * 全局变量
41  * 1. 句柄列表
42  * 2. 句柄数
43  */
44 static int read_handles(const char *path, char handles[][MAX_HANDLE_LEN]) {
45     FILE *f = fopen(path, "r");
46     if (!f) {
47         fprintf(stderr, "Cannot open handle list: %s\n", path);
48         return -1;
49     }
50
51     int count = 0;
52     char buf[256];
53     while (fgets(buf, sizeof(buf), f) && count < MAX_HANDLES) {
54         char *s = buf;
55         while (*s == ' ' || *s == '\t' || *s == '\r') s++;
56         int len = (int)strlen(s);
57         while (len > 0 && (s[len-1] == '\n' || s[len-1] == '\r' || s[len-1] == ' ' || s[len-1] == '\t'))
58             s[--len] = '\0';
59         if (len > 0 && len < MAX_HANDLE_LEN) {
60             memmove(handles[count], s, (size_t)len + 1);
61             count++;
62         }
63     }
64     fclose(f);
65     return count;
66 }
67
```

```

68 int main(int argc, char *argv[]) {
69     /* ■■■■ sample_users.txt■■■■■■■■■■■■■■■■■■■■ */
70     const char *handle_file = "sample_users.txt";
71     if (argc > 1) handle_file = argv[1];
72
73     printf("=== Codeforces Crawler 2025 ===\n");
74     printf("Handle list: %s\n\n", handle_file);
75
76     /* ■■ 1: ■■■■■■ */
77     char handles[MAX_HANDLES][MAX_HANDLE_LEN] = {{0}};
78     int handle_count = read_handles(handle_file, handles);
79     if (handle_count <= 0) {
80         fprintf(stderr, "No valid handles found in %s\n", handle_file);
81         return 1;
82     }
83     printf("Found %d handle(s)\n\n", handle_count);
84
85     /* ■■■■■■■■■■ */
86     mkdir_p(DATA_DIR);
87     mkdir_p(OUTPUT_DIR);
88
89     /* ■■■ CF API ■■■■2 ■■■■ */
90     RateLimiter rl;
91     rate_limiter_init(&rl);
92
93     /* ■■ 2: ■■■■■■■■■■■■■■■■■■■■ */
94     printf("[1/5] Fetching contest list...\n");
95     if (cf_fetch_contest_list(DATA_DIR "/contest_list.json", &rl) != 0) {
96         fprintf(stderr, "Failed to fetch contest list\n");
97     }
98
99     /* ■■ 3: ■■■■■■■■■■ */
100    cJSON *users_arr = cJSON_CreateArray();
101    if (!users_arr) {
102        fprintf(stderr, "Memory error\n");
103        return 1;
104    }
105
106    for (int i = 0; i < handle_count; i++) {
107        const char *handle = handles[i];
108        printf("\n--- Processing: %s (%d/%d) ---\n", handle, i + 1, handle_count);
109
110        char path[1024];
111
112        /* 3a: ■■■■■■■■■rating■■■■■■■■■ */
113        printf("[1/3] Fetching user info...\n");
114        snprintf(path, sizeof(path), "%s/%s_user.json", DATA_DIR, handle);
115        if (cf_fetch_user_info(handle, path, &rl) != 0) {
116            fprintf(stderr, " Skipping %s (failed to fetch user info)\n", handle);
117            continue;
118        }
119
120        /* 3b: ■■ rating ■■■■■■■■■■ */
121        printf("[2/3] Fetching rating history...\n");
122        snprintf(path, sizeof(path), "%s/%s_rating.json", DATA_DIR, handle);
123        if (cf_fetch_user_rating(handle, path, &rl) != 0) {
124            fprintf(stderr, " Skipping %s (failed to fetch rating history)\n", handle);
125            continue;
126        }
127
128        /* 3c: ■■■■■■■■■■■■■■■■■■■■■■ */
129        printf("[3/3] Fetching submissions...\n");
130        snprintf(path, sizeof(path), "%s/%s_status.json", DATA_DIR, handle);
131        if (cf_fetch_user_status(handle, path, &rl) != 0) {
132            fprintf(stderr, " Warning: failed to fetch submissions for %s\n", handle);
133        }
134
135        /* 3d: ■■■■■■■■■■■■■■■ JSON */
136        printf(" Analyzing data...\n");
137        cJSON *summary = build_user_summary(handle, DATA_DIR, OUTPUT_DIR, 0);
138        if (summary) {
139            cJSON_AddItemToArray(users_arr, summary);

```

```
140         printf(" Analysis complete.\n");
141     } else {
142         fprintf(stderr, " Failed to analyze data for %s\n", handle);
143     }
144 }
145
146 /* ■■■ 4: ■■■■■■■■ HTML ■■■ */
147 printf("\n=== Generating HTML reports ===\n");
148 if (cJSON_GetArraySize(users_arr) > 0) {
149     if (generate_all_html(users_arr, TEMPLATE_PATH, OUTPUT_DIR) == 0) {
150         printf("\nDone! Open output/index.html in your browser.\n");
151     } else {
152         fprintf(stderr, "HTML generation failed.\n");
153     }
154 } else {
155     fprintf(stderr, "No user data to generate.\n");
156 }
157
158 cJSON_Delete(users_arr);
159 return 0;
160 }
```